

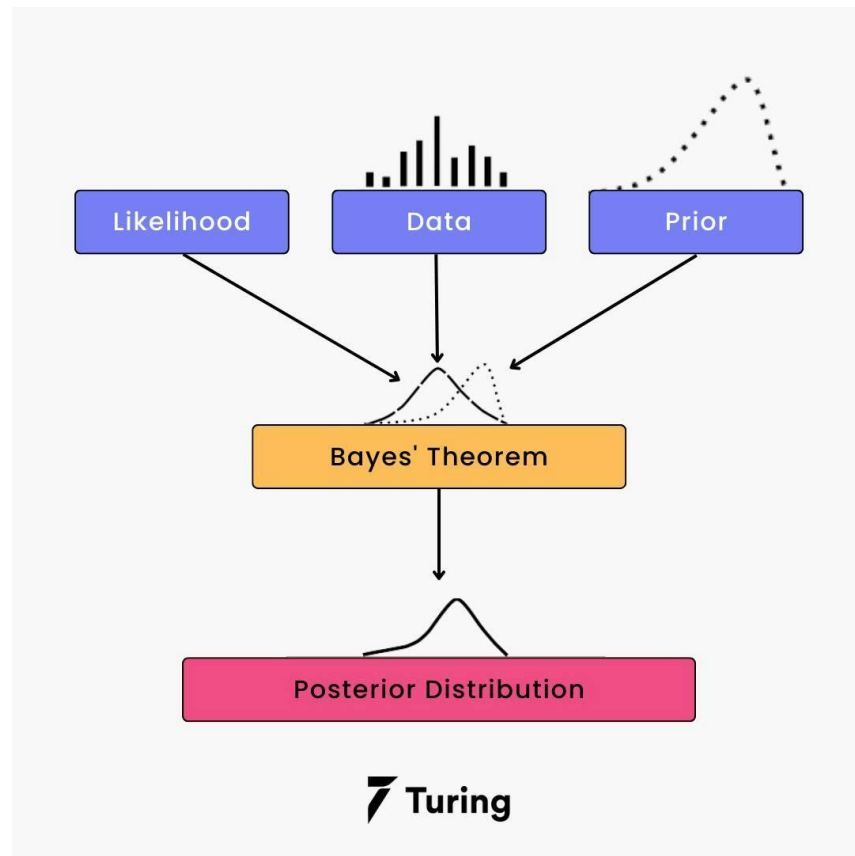
UNIT 2

Bayesian Theory

1. Introduction

In pattern recognition, data obtained from real-world sources such as images, handwriting, or speech is often uncertain and noisy. The same pattern may appear different under varying conditions such as lighting, writing style, or background noise. Because of this uncertainty, it is not always possible to make a perfectly certain decision.

Bayesian theory provides a mathematical and conceptual framework to handle this uncertainty by using probability. It allows a system to update its belief about a class after observing data.



2. Basic Idea of Bayesian Theory

Bayesian theory is based on the principle that **probability represents degree of belief**. Instead of treating probability as a fixed frequency only, Bayesian theory treats it as a measure of uncertainty about an event.

In simple words, Bayesian theory answers the question:

“Given the observed data, how likely is it that the data belongs to a particular class?”

The theory continuously updates knowledge as new evidence becomes available.

3. Core Components of Bayesian Theory

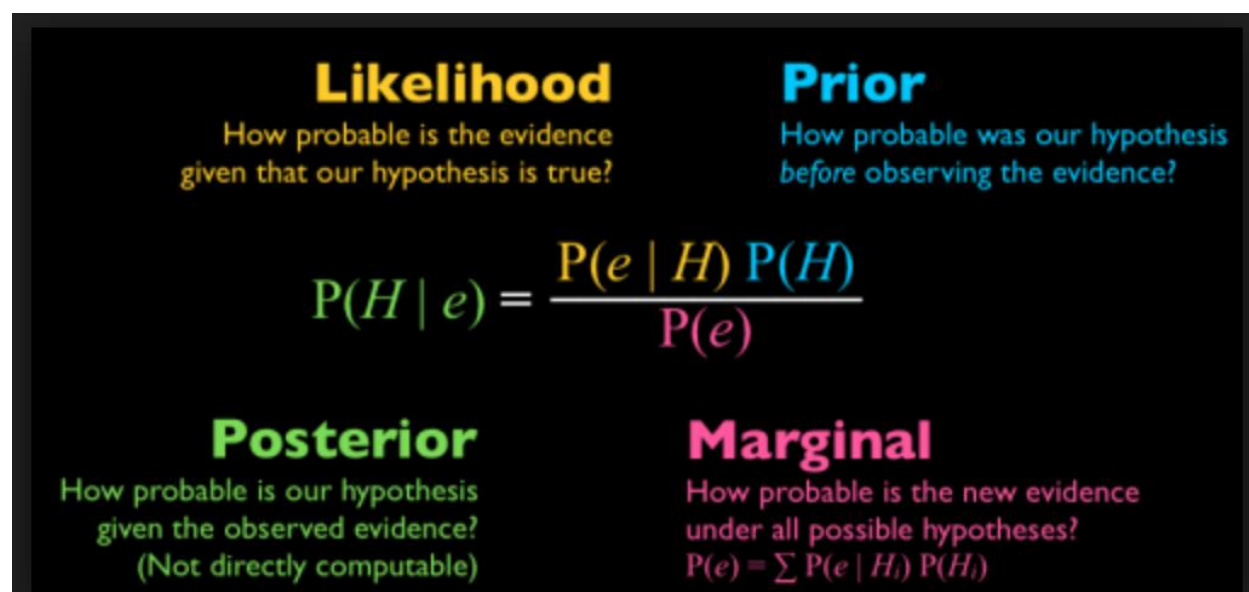
Bayesian theory relies on three main probability concepts: **prior probability**, **likelihood**, and **posterior probability**.

3.1 Prior Probability

The **prior probability** represents the initial belief about a class before observing any data. It is based on previous knowledge or experience.

For example, in handwritten digit recognition, some digits may appear more frequently than others in the training dataset. Before seeing a new image, the system already has an idea about how common each digit is. This information is captured by the prior probability.

Prior probability helps incorporate real-world knowledge into pattern recognition systems.



3.2 Likelihood

The **likelihood** represents how well the observed data matches a particular class. It measures the probability of observing the given data assuming that the data belongs to a specific class.

For example, given a handwritten image, the likelihood measures how closely the image resembles the typical writing style of digit “5” or digit “8”. Even if two digits look similar, their likelihood values may differ based on feature similarity.

Likelihood captures the relationship between **features and classes**.

3.3 Posterior Probability

The **posterior probability** is the updated belief about a class after observing the data. It is obtained by combining the prior probability and the likelihood.

In pattern recognition, the posterior probability answers the most important question:
“After seeing the data, which class is most probable?”

Classification decisions are usually made by comparing posterior probabilities of different classes.

4. Bayes’ Theorem

Bayesian theory is mathematically formalized using **Bayes’ theorem**, which describes how prior probability is updated using observed data.

Conceptually, Bayes’ theorem states that:

- Start with prior knowledge
- Observe new data
- Update belief to obtain posterior probability

Although the mathematical formula is important, the **conceptual understanding** is more critical for pattern recognition applications.

5. Bayesian Classification Process

In a Bayesian classification system, the following steps are followed:

First, prior probabilities of all classes are estimated using training data.

Second, likelihood models are learned to describe how features behave for each class.

Third, when a new sample is observed, posterior probabilities are computed for all classes.

Finally, the class with the highest posterior probability is selected.

This systematic approach makes Bayesian classifiers robust and interpretable.

6. Example: Handwriting Recognition Using Bayesian Theory

Consider a handwritten digit that looks similar to “3” and “8”.

The system first considers the prior probabilities of digits “3” and “8”. If digit “3” appears more frequently in the dataset, it has a higher prior probability.

Next, the system evaluates how well the extracted features match the typical patterns of “3” and “8”. This gives the likelihood values.

By combining prior probability and likelihood, the system computes posterior probabilities.

The digit is finally classified as the class with the higher posterior probability.

Thus, Bayesian theory allows the system to make a rational decision even when patterns overlap.

7. Example: Face Recognition Using Bayesian Theory

In face recognition, Bayesian theory is used to estimate the probability that a given face image belongs to a particular person. Prior probabilities may represent how often a person appears in the database, while likelihood measures how well facial features match that person. The posterior probability helps decide the identity of the face.

8. Advantages of Bayesian Theory in Pattern Recognition

Bayesian theory provides a principled way to deal with uncertainty. It allows incorporation of prior knowledge and adapts naturally as new data becomes available. Bayesian methods are also flexible and can be applied to various domains such as image recognition, handwriting recognition, and speech recognition.

9. Limitations of Bayesian Theory

Bayesian theory requires accurate estimation of prior probabilities and likelihood functions. If these estimates are poor, classification performance may degrade. In high-dimensional data, probability estimation can become computationally complex.

Decision Theory

1. Introduction

In pattern recognition, a system often needs to choose between multiple possible classes based on imperfect or noisy data. For example, a face image may be blurred, a handwritten digit may be ambiguous, or a speech signal may be affected by background noise. In such cases, making a decision based only on similarity or probability may not be sufficient.

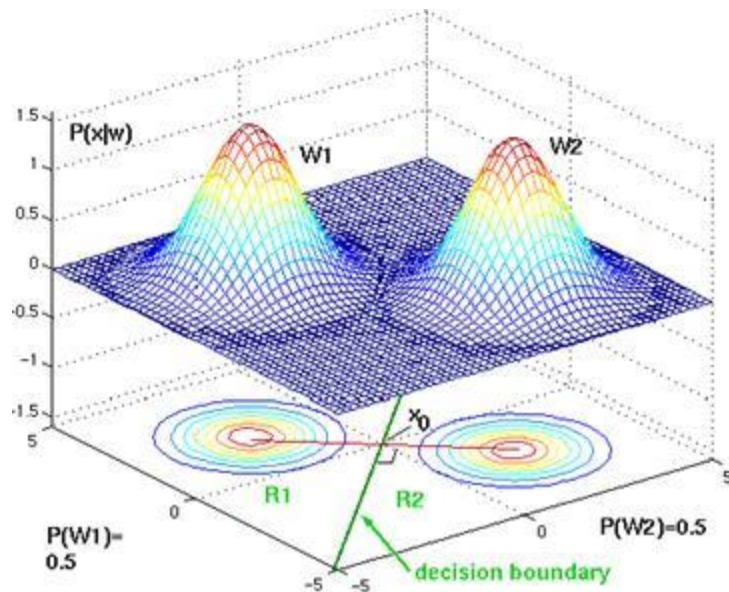
Decision theory provides a principled framework for making the **best possible decision** by considering not only probabilities but also the **consequences of decisions**.

2. Basic Idea of Decision Theory

Decision theory is concerned with **choosing an action** when the true class of a pattern is uncertain. The key idea is that **not all errors are equally serious**. Some wrong decisions may be harmless, while others may have severe consequences.

In simple words, decision theory answers the question:

“Given the uncertainty and the cost of mistakes, what decision should be made?”



3. Components of Decision Theory

Decision theory is built on three conceptual components:

First, there are **possible classes or states**, such as different digits, people, or speech commands. Second, there are **possible decisions or actions**, such as accepting, rejecting, or assigning a class label.

Third, there is a **loss (or cost)** associated with each decision when the true class is known.

The goal of decision theory is to select the decision that results in the **minimum expected loss**, also known as **minimum risk**.

4. Loss and Risk

Loss

Loss represents the **penalty** for making a particular decision when the true class is something else. For example, misclassifying a digit “1” as “7” may have a small loss, while misclassifying an unauthorized person as authorized may have a very high loss.

Risk

Risk is the **average loss expected**, taking into account the uncertainty about the true class. A good decision is one that **minimizes this expected risk**, not necessarily one that is simply the most probable.

5. Decision Rule in Decision Theory

Decision theory states that the system should choose the decision that **minimizes expected risk**. This rule allows the system to trade off probability against cost. As a result, the final decision may differ from the class with the highest probability if the cost of being wrong is very high.

6. Worked Example: Face Recognition (Security System)

Consider a face recognition system at the entrance of a secure building. The system must decide whether to **allow entry** or **deny entry**.

There are two possible true states:

- The person is **authorized**
- The person is **unauthorized**

There are two possible decisions:

- **Grant access**
- **Deny access**

Now consider the losses:

- Granting access to an unauthorized person has **very high loss** (security breach).
- Denying access to an authorized person has **moderate loss** (inconvenience).
- Correct decisions have **low or zero loss**.

Even if the probability that a person is authorized is slightly higher than unauthorized, decision theory may still choose to **deny access** if the cost of a false acceptance is much higher. This shows that decision theory does not simply choose the most probable class; it chooses the **safest decision** in terms of expected loss.

7. Another Example: Medical Diagnosis

Consider a system diagnosing a disease:

- Misdiagnosing a healthy person as sick may cause stress but is reversible.
- Misdiagnosing a sick person as healthy may delay treatment and cause serious harm.

Decision theory assigns higher loss to the second error. Therefore, the system may choose a decision rule that favors **early detection**, even if it increases the number of false alarms. This behavior is rational under decision theory because it minimizes overall risk.

8. Decision Boundaries in Pattern Recognition

Using decision theory, the feature space is divided into **decision regions**. Each region corresponds to a decision that minimizes expected risk.

These regions are separated by **decision boundaries**, which may shift depending on the loss values. When costs change, the decision boundary changes—even if probabilities remain the same.

9. Importance of Decision Theory in Pattern Recognition

Decision theory is important because it:

- Handles uncertainty explicitly
- Incorporates real-world costs of errors
- Produces rational and optimal decisions
- Is suitable for safety-critical applications
- Works with probabilistic classifiers

It is widely used in applications such as security systems, medical diagnosis, fraud detection, and speech command recognition.

Decision Theory

Mathematical Formulation

Core Idea

Decision theory is based on **expected risk minimization**.

Key Terms

- $\omega_i \rightarrow$ true class
 - $\alpha_j \rightarrow$ decision/action
 - $P(\omega_i | \mathbf{x}) \rightarrow$ probability that sample $\mathbf{x}$ belongs to class ω_i
 - $\lambda(\alpha_j | \omega_i) \rightarrow$ loss if decision α_j is taken when true class is ω_i
-

Expected Risk (Main Formula)

$$R(\alpha_j | \mathbf{x}) = \sum_{i=1}^c \lambda(\alpha_j | \omega_i) P(\omega_i | \mathbf{x})$$

Meaning in words:

Risk of a decision = sum of (probability $\times$ loss) over all classes

Decision Rule

Choose α_j such that $R(\alpha_j | \mathbf{x}) = \min_k R(\alpha_k | \mathbf{x})$

Exam line:

Decision theory selects the action that minimizes expected risk.

Why This Formula Makes Sense (Intuition)

- Probability alone is **not enough**
- Loss alone is **not enough**
- Their **product** gives a fair measure of danger

This is why decision theory is **more practical** than pure Bayesian classification.

Numerical Practice Examples

Example 1: Binary Classification (Face Recognition)

Two classes:

- ω_1 = Authorized
- ω_2 = Unauthorized

Two decisions:

- α_1 = Allow
- α_2 = Deny

Given probabilities:

- $P(\omega_1 | x) = 0.7$
- $P(\omega_2 | x) = 0.3$

Loss matrix:

Decision \ True class Authorized Unauthorized

Allow (α_1) 0 100

Deny (α_2) 5 0

Risk Calculation

Risk of Allow (α_1):

$$R(\alpha_1|x) = (0 \times 0.7) + (100 \times 0.3) = 30$$

Risk of Deny (α_2):

$$R(\alpha_2|x) = (5 \times 0.7) + (0 \times 0.3) = 3.5$$

Decision

Since $3.5 < 30$,

Decision = DENY

Even though authorized is more probable, decision theory chooses the safer action.

Example 2: Handwriting Recognition (Digit 3 vs 8)

Probabilities:

- $P(3 | x) = 0.55$
- $P(8 | x) = 0.45$

Loss:

- Misclassifying 3 as 8 $\rightarrow$ loss = 2
- Misclassifying 8 as 3 $\rightarrow$ loss = 10

Risk of choosing 3:

[
 $R(3) = 10 * 0.45 = 4.5$
]

Risk of choosing 8:

[
 $R(8) = 2 * 0.55 = 1.1$
]

Decision

Choose **8**, because **1.1 < 4.5**

Example 3: Medical Diagnosis (Classic Exam Question)

Classes:

- $\omega_1 = \text{Disease}$
- $\omega_2 = \text{Healthy}$

Probabilities:

- $P(\text{Disease} | x) = 0.4$
- $P(\text{Healthy} | x) = 0.6$

Loss:

- Miss disease $\rightarrow$ loss = 50
 - False alarm $\rightarrow$ loss = 5
-

Risk of saying Healthy:

[

$$R = 50 * 0.4 = 20$$

]

Risk of saying Disease:

[

$$R = 5 * 0.6 = 3$$

]

Decision

Diagnose **Disease**

This explains why medical systems prefer **early detection**.

Classifiers in Pattern Recognition

Definition

A classifier is an algorithm or mathematical model that assigns an input pattern to one of several predefined classes based on its features.

In simple terms, a classifier is a decision-making system that identifies the category to which new data belongs.

Basic Idea of a Classifier

In pattern recognition, data is first converted into numerical features. The classifier studies these features and decides which class the data belongs to.

Example:

- Input: Email message
 - Output: Spam or Not Spam
-

Working of a Classifier

The working of a classifier generally involves two main phases.

1. Training Phase

- The classifier learns from labeled data.
- It studies the relationship between features and class labels.
- A mathematical model is created.

2. Testing Phase

- The trained model is used to classify new unknown data.
 - The classifier predicts the class label.
-

Example to Explain to Students

Suppose we want to classify fruits based on features.

Feature 1	Feature 2	Class
Red color	Round shape	Apple
Yellow color	Long shape	Banana

If a new fruit has red color and round shape, the classifier predicts it as an apple.

Types of Classifiers

1. Statistical Classifiers

These classifiers use probability and statistical methods.

Examples:

- Bayes classifier
- Maximum Likelihood classifier

2. Distance-Based Classifiers

These classifiers assign a class based on similarity or distance between patterns.

Example:

- K-Nearest Neighbor (KNN)

3. Linear Classifiers

These classifiers separate classes using straight-line or linear decision boundaries.

Examples:

- Perceptron
- Linear Discriminant Analysis

4. Non-Linear Classifiers

These classifiers separate complex data using curved boundaries.

Examples:

- Support Vector Machine (SVM)
- Neural Networks

Decision Boundary

A decision boundary is a surface that separates different classes in feature space.

Types:

- Linear boundary (straight line or plane)
 - Non-linear boundary (curved surface)
-

Characteristics of a Good Classifier

A good classifier should:

- Produce high accuracy
 - Handle noise and uncertainty
 - Work efficiently with large data
 - Generalize well to new data
-

Applications of Classifiers

Classifiers are used in:

- Face recognition
 - Medical diagnosis
 - Speech recognition
 - Spam detection
 - Document classification
 - Fraud detection
-

Conclusion

Classifiers are the core component of pattern recognition systems. They analyze extracted features and make final decisions about class membership.

Discriminant Functions in Pattern Recognition

Definition

A discriminant function is a mathematical function used by a classifier to assign an input pattern to one of several classes. It helps in separating different classes by evaluating a numerical value for each class and selecting the class with the highest value.

Basic Idea

In pattern recognition, each input pattern is represented by a feature vector. A discriminant function uses this feature vector to determine which class the pattern belongs to.

If $g_i(x)$ represents the discriminant function for class (ω_i) , then the decision rule is:

Assign pattern x to class ω_i if $g_i(x) > g_j(x)$ for all $j \neq i$

Purpose of Discriminant Functions

- To separate different classes in feature space
 - To provide decision boundaries between classes
 - To simplify classification problems using mathematical models
-

Types of Discriminant Functions

1. Linear Discriminant Function

A linear discriminant function has the form:

$$[\begin{aligned} g(x) &= w^T x + w_0 \end{aligned}]$$

where:

- (w) is the weight vector
- (x) is the feature vector
- (w_0) is the bias or threshold

Characteristics:

- Produces linear decision boundaries
- Suitable when classes are linearly separable
- Computationally simple

2. Quadratic Discriminant Function

A quadratic discriminant function has the form:

$$g(x) = x^T A x + w^T x + w_0$$

where:

- (A) is a matrix defining quadratic terms

Characteristics:

- Produces curved decision boundaries
- Handles more complex data distributions

3. Non-Linear Discriminant Function

These functions are used when data cannot be separated using linear or quadratic boundaries.

Examples:

- Neural networks
- Kernel-based classifiers

Characteristics:

- Capable of modeling complex relationships
- Higher computational complexity

Decision Boundary

A decision boundary is formed where two discriminant functions have equal values.

Mathematically:

$$g_i(x) = g_j(x)$$

This boundary separates feature space into different class regions.

Example

Suppose two classes are separated using a linear discriminant function. The classifier calculates discriminant values for each class and assigns the pattern to the class with the highest value.

Advantages of Discriminant Functions

- Provides systematic decision rules
 - Simplifies classification problems
 - Helps in visualizing class boundaries
-

Applications

Discriminant functions are widely used in:

- Face recognition
 - Speech recognition
 - Medical diagnosis
 - Document classification
 - Object recognition
-

Conclusion

Discriminant functions play an essential role in classification by providing mathematical rules to separate classes. The choice of linear, quadratic, or nonlinear discriminant functions depends on the complexity of data and application requirements.

Comparison of Discriminant Function and Classifier

Discriminant functions and classifiers are closely related concepts in pattern recognition. A discriminant function is a mathematical tool used inside a classifier to make decisions, while a classifier is a complete system that assigns class labels to input patterns.

Discriminant Function

Definition

A discriminant function is a mathematical function used to separate classes by computing a numerical value for each class based on the feature vector of the input pattern.

Purpose

- To provide decision rules for classification
- To form decision boundaries between classes
- To measure how strongly a pattern belongs to a class

Example

Linear discriminant function:

$$g(x) = w^T x + w_0$$

Classifier

Definition

A classifier is an algorithm or model that assigns input patterns to predefined classes using training data and decision rules.

Purpose

- To perform the complete classification process
- To learn from labeled data
- To predict class labels for new data

Example

- Bayes classifier

- K-Nearest Neighbor
- Support Vector Machine
- Neural Networks

Key Differences Between Discriminant Function and Classifier

Aspect	Discriminant Function	Classifier
Nature	Mathematical function	Complete decision-making model
Role	Provides decision rule	Performs full classification process
Function	Separates classes using mathematical expression	Uses training data and algorithms to assign labels
Complexity	Usually simple mathematical expression	May involve multiple algorithms and training phases
Output	Numerical score for each class	Final class label
Usage	Component of classifier	Complete recognition system
Learning Ability	Does not learn independently	Learns from training data

Relationship Between Discriminant Function and Classifier

- A classifier uses one or more discriminant functions.
- Discriminant functions help classifiers determine decision boundaries.
- Without discriminant functions, many classifiers cannot perform classification.

Conclusion

Discriminant functions and classifiers are interrelated concepts in pattern recognition. The discriminant function acts as a mathematical decision tool, whereas the classifier is a complete system that learns from data and assigns class labels.

Decision Surface in Pattern Recognition

Definition

A decision surface is a boundary in the feature space that separates different classes in a classification problem. It is formed by the points where two or more discriminant functions produce equal values.

Basic Concept

In pattern recognition, data is represented as feature vectors in a multidimensional space called feature space. A classifier divides this feature space into different regions corresponding to different classes.

The boundary separating these regions is called the decision surface.

Mathematical Representation

A decision boundary is formed where two discriminant functions have equal values.

Mathematically:

$$g_i(x) = g_j(x)$$

This boundary separates feature space into different class regions.

Types of Decision Surfaces

1. Linear Decision Surface

A linear decision surface is formed when the discriminant function is linear.

Example:

$$\begin{bmatrix} g(x) = w^T x + w_0 \end{bmatrix}$$

Characteristics:

- Produces straight line in two-dimensional space
 - Produces plane in three-dimensional space
 - Produces hyperplane in higher dimensions
 - Used when classes are linearly separable
-

2. Non-Linear Decision Surface

A nonlinear decision surface is formed when discriminant functions are nonlinear.

Characteristics:

- Produces curved boundaries
 - Used when data cannot be separated using straight lines
 - Provides better classification for complex data
-

Decision Regions

A decision region is the area in feature space assigned to a particular class. The decision surface divides the feature space into multiple decision regions.

Importance of Decision Surfaces

Decision surfaces are important because they:

- Define classification boundaries
 - Help visualize separation between classes
 - Determine classifier performance
 - Help understand class distribution
-

Example

Consider a two-class problem where data points representing two classes are plotted on a graph. The classifier draws a boundary line separating the classes. This boundary line is the decision surface.

Applications

Decision surfaces are used in:

- Face recognition systems
 - Medical diagnosis
 - Speech recognition
 - Object classification
 - Fraud detection
-

Conclusion

A decision surface is a critical concept in classification that separates feature space into regions representing different classes. The nature of the decision surface depends on the type of classifier and the complexity of the data.

Parameter Estimation Methods in Pattern Recognition

Definition

Parameter estimation is the process of determining the values of unknown parameters of a probability distribution or mathematical model using observed data. These parameters are used to describe the statistical properties of different classes in pattern recognition.

Need for Parameter Estimation

In statistical pattern recognition, classifiers require probability distributions to model class data. Since the true distributions are usually unknown, parameters such as mean and variance must be estimated from training data.

Parameter estimation helps in:

- Modeling class distributions
- Improving classification accuracy
- Designing statistical classifiers

Types of Parameter Estimation Methods

1. Maximum Likelihood Estimation (MLE)

Definition

Maximum Likelihood Estimation determines parameter values that maximize the probability of observing the given training data.

Concept

MLE assumes that the best parameter value is the one that makes the observed data most likely.

Example

For a Gaussian distribution:

- Mean is estimated using the average of training samples
- Variance is estimated using sample variance

Advantages

- Simple and widely used
- Provides good estimates with large datasets

Limitations

- Does not consider prior knowledge
- Sensitive to small datasets

Example 1: Estimating Mean and Variance of Gaussian Distribution

Maximum Likelihood Estimation (MLE) is a method. It works for any probability distribution.

We can apply MLE to:

- Bernoulli distribution
- Binomial distribution
- Poisson distribution

- Gaussian distribution
- Exponential distribution

Gaussian is not mandatory.

Given Data

Suppose we have the following training samples:

$x = \{4, 6, 8, 10, 12\}$

Assume data follows a **Gaussian (Normal) distribution**.

We want to estimate:

- Mean μ
- Variance σ^2

using MLE.

Step 1: Formula for MLE of Mean

For Gaussian distribution:

$$\mu_{MLE} = 1/n \sum x_i$$

Number of samples:

$$n = 5$$

Sum of data:

$$4 + 6 + 8 + 10 + 12 = 40$$

So,

$$\mu_{MLE} = 40/5 = 8$$

Step 2: Formula for MLE of Variance

$$\sigma^2_{MLE} = 1/n \sum (x_i - \mu)^2$$

Compute squared differences:

$$(4-8)^2=16$$

$$(6-8)^2=4$$

$$(8-8)^2=0$$

$$(10-8)^2=4$$

$$(12-8)^2=16$$

Sum:

$$16+4+0+4+16= 40$$

Now divide by n:

$$\sigma^2_{\text{MLE}} = 40/5 = 8$$

MLE tells us that:

- The most likely mean of this dataset is 8.
- The spread (variance) of data is 8.

These values make the observed data most probable under Gaussian assumption.

Example 2: Simple Coin Toss (Bernoulli MLE)

Suppose a coin is tossed 12 times and we get:

Heads = 9

Tails = 3

We want to estimate probability of heads p .

MLE formula for Bernoulli:

$P_{MLE} = \text{Number of Heads} / \text{Total Tosses}$

$p_{MLE} = 9/12 = 0.75$

So estimated probability of heads is 0.75.

MLE estimate is simply:

Observed frequency = Estimated probability

2. Bayesian Estimation

Definition

Bayesian estimation incorporates prior knowledge about parameters and updates this knowledge using observed data.

Concept

Bayesian estimation treats parameters as random variables and calculates posterior probability using Bayes theorem.

Advantages

- Uses prior information
- More reliable when data is limited
- Provides probabilistic interpretation

Limitations

- Computationally complex
- Requires prior distribution selection

3. Maximum A Posteriori (MAP) Estimation

Definition

MAP estimation determines parameter values that maximize posterior probability.

Concept

MAP combines prior probability and likelihood information.

Relationship

- MAP becomes MLE when prior probability is uniform.

Numerical Example: Maximum A Posteriori (MAP) Estimation

Example: Coin Toss with Prior Knowledge

Suppose:

We toss a coin 10 times

Observed result:

Heads = 7

Tails = 3

We want to estimate probability of heads p

Step 1: MLE Estimation (For Comparison)

MLE formula for Bernoulli:

$P_{MLE} = \text{Number of Heads} / \text{Total Tosses}$

$p_{MLE} = 7/10 = 0.7$

So estimated probability of heads is 0.7

MLE ignores prior knowledge.

Step 2: Now Introduce Prior (MAP Idea)

Suppose from previous experience we believe:

The coin is slightly biased toward tails.

We assume a prior distribution:

Beta(2, 4)

This means:

- Prior heads = 2
- Prior tails = 4

(Think of it as if we already observed 2 heads and 4 tails earlier.)

Step 3: MAP Formula for Bernoulli with Beta Prior

For Beta prior:

$$P_{\text{MAP}} = (\text{Heads} + \alpha - 1) / (\text{Total Tosses} + \alpha + \beta - 2)$$

Where:

$$\alpha = 2, \beta = 4$$

Step 4: Substitute Values

$$p_{\text{MAP}} = (7 + 2 - 1) / (10 + 2 + 4 - 2)$$

$$p_{\text{MAP}} \approx 0.57$$

Final Results

MLE estimate:

0.7

MAP estimate:

0.57

Important Teaching Point

MLE says:

“Only use current data.”

MAP says:

“Use current data + prior belief.”

Because prior favored tails, MAP pulls estimate toward tails.

Steps in Parameter Estimation

1. Collect training data
 2. Assume probability distribution model
 3. Select estimation method
 4. Estimate distribution parameters
 5. Use estimated parameters in classifier design
-

Applications of Parameter Estimation

Parameter estimation is used in:

- Bayesian classifiers
- Gaussian distribution modeling
- Speech recognition
- Medical diagnosis
- Image classification

Importance in Pattern Recognition

Parameter estimation is important because:

- It helps in understanding data distribution
 - It improves decision-making accuracy
 - It supports statistical modeling and classification
-

Conclusion

Parameter estimation methods are essential for statistical pattern recognition as they determine the unknown parameters required for modeling class distributions. Maximum Likelihood, Bayesian estimation, and MAP estimation are commonly used techniques depending on data availability and prior knowledge.

Below are **clear, classroom-ready notes** on **Markov Model and Hidden Markov Model (HMM)** with diagrams for student understanding.

Markov Model (Markov Chain)

Definition

A Markov Model is a stochastic model that describes a system which moves between states, where the probability of the next state depends only on the current state.

This is called the **Markov Property**.

$P = (S_t, S_{t-1}, S_{t-2}, \dots)$

Key Characteristics

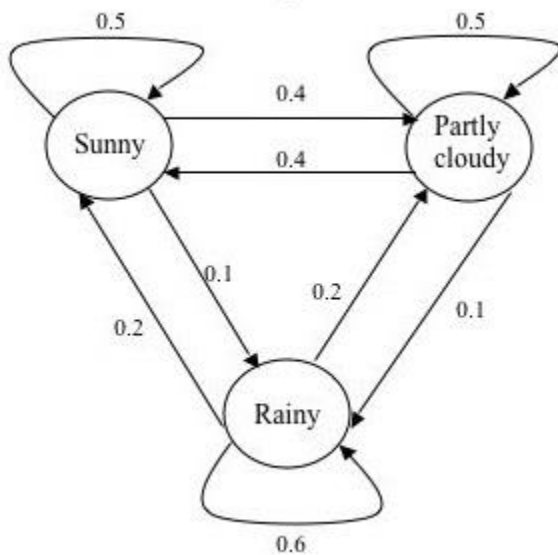
- States are directly observable.

- Only transition probabilities are involved.
 - No hidden layer.
 - Memoryless property (depends only on present state).
-

Components of Markov Model

1. Set of states
[
S = {S1, S2, ..., SN}
]
 2. Transition Probability Matrix
[
A = [a{ij}]
]
where
[
a{ij} = P(Sj | Si)
]
-

State Transition Diagram (Markov Model)



Explanation

- Circles represent states.
 - Arrows represent transition probabilities.
 - States are directly observable.
-

Example

Weather model:

States:

Rainy (R), Sunny (S)

If today is Rainy:

$$P(R \rightarrow R) = 0.7$$

$$P(R \rightarrow S) = 0.3$$

You directly observe weather.

Applications

- Weather prediction
 - Stock price modeling
 - Queue systems
 - Population modeling
-

Hidden Markov Model (HMM)

Definition

A Hidden Markov Model (HMM) is a statistical model used to represent systems that evolve over time where the actual states of the system are not directly observable. Instead, we observe outputs or signals generated by those hidden states. The model is based on the Markov property, which states that the future state depends only on the present state and not on past states. In an HMM, there are two layers: a hidden state layer and an observation layer. The hidden states transition from one to another with certain probabilities (transition probabilities), and each state produces observable outputs according to emission probabilities. HMMs are widely used in pattern recognition tasks involving sequential data such as speech recognition, handwriting recognition, natural language processing, and bioinformatics. They are powerful because they allow us to infer hidden patterns from observable data.

A Hidden Markov Model is a statistical model where:

- The system has hidden states.
 - We cannot observe states directly.
 - We observe outputs generated by states.
-

Key Idea

Hidden State $\rightarrow$ Generates $\rightarrow$ Observation

Components of HMM

An HMM is defined by:

1. States

$$S = \{S_1, S_2, \dots, S_N\}$$

2. Transition Probability Matrix (A)

$$a_{ij} = P(S_j | S_i)$$

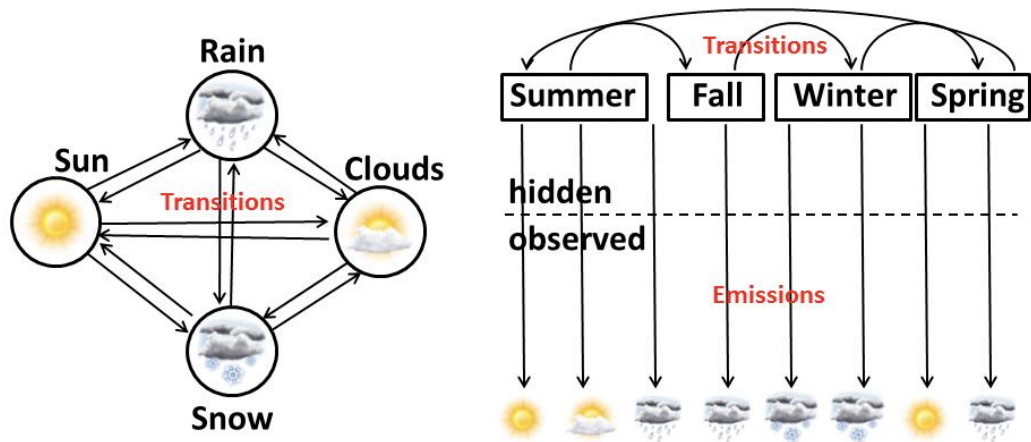
3. Emission Probability Matrix (B)

$$b_j(k) = P(O_k | S_j)$$

4. Initial Probability Vector (π)

$$\pi_i = P(S_i \text{ at } t = 1)$$

HMM Diagram



Explanation

- Top layer = Hidden states.
- Bottom layer = Observations.
- Arrows between states = Transition probabilities.
- Arrows from states to observations = Emission probabilities.

Example

Weather is hidden.

States:

Rainy, Sunny

Observations:

Umbrella, No Umbrella

You observe umbrella and infer weather.

Three Main Problems in HMM

1. Evaluation
Compute probability of observation sequence.
(Forward Algorithm)
 2. Decoding
Find most likely hidden state sequence.
(Viterbi Algorithm)
 3. Learning
Estimate model parameters.
(Baum-Welch Algorithm)
-

First Understand the Three Things Clearly

In Hidden Markov Model there are **three different concepts**:

- 1 Hidden States
- 2 Transition Probabilities
- 3 Emission (Observation) Probabilities

Let's separate them properly.

1 Hidden States (What We Cannot See)

These are the real conditions of the system.

Example (Weather):

- Rainy (R)
- Sunny (S)

We cannot directly see the true state.

That is why it is called **Hidden**.

2 Observations (What We Actually See)

These are visible signals produced by hidden states.

Example:

- Umbrella (U)
- No Umbrella (N)

We do not see weather directly.

We only see people carrying umbrellas.

3 Transition Probabilities (State $\rightarrow$ State)

This tells how states change over time.

Example:

$$P(R \rightarrow S) = 0.3 \quad P(R \rightarrow S) = 0.3 \quad P(R \rightarrow S) = 0.3$$

Meaning:

If today is Rainy, tomorrow becomes Sunny with probability 0.3.

Transition = movement between hidden states.

4 Emission Probabilities (State $\rightarrow$ Observation)

This tells what observation a state produces.

Example:

$$P(U|R)=0.9$$

Meaning:

If state is Rainy, probability of seeing umbrella is 0.9.

Emission = hidden state generating visible output.

One Simple Story

Imagine:

You are inside a room.

You cannot see weather.

You only see people entering.

If many people carry umbrellas → probably raining.

Weather = hidden state

Umbrella = observation

Weather changing over days = transition

Weather producing umbrella behavior = emission

Difference Between Markov Model and HMM

Aspect	Markov Model	Hidden Markov Model
State visibility	Visible	Hidden
Observations	States themselves	Separate observation layer
Probabilities	Transition only	Transition + Emission
Example	Direct weather observation	Weather inferred from umbrella

Aspect	Markov Model	Hidden Markov Model
Complexity	Simpler	More complex

Dimensionality Reduction Methods

1. Introduction

Dimensionality reduction is the process of reducing the number of input variables (features) in a dataset while preserving the most important information. In pattern recognition and machine learning, datasets may contain a large number of features, which can increase computational complexity and sometimes reduce model performance. Dimensionality reduction helps simplify the data, remove redundant information, and improve efficiency of algorithms.

2. Need for Dimensionality Reduction

Dimensionality reduction is required because:

- High dimensional data increases computational cost.
- Many features may be redundant or irrelevant.
- It helps improve model performance.
- It reduces storage requirements.
- It helps visualize high dimensional data.

This problem is often called the **curse of dimensionality**.

3. Types of Dimensionality Reduction Methods

Dimensionality reduction techniques are broadly classified into two categories:

1. Feature Selection

2. Feature Extraction

1. Feature Selection

Definition

Feature selection is the process of selecting a subset of the original features that are most relevant for the learning task.

In this method, no new features are created. Only the most important features are selected.

Example

Suppose a dataset has 10 features:

Height, Weight, Age, Salary, City, Phone Number, Email, Education, Blood Group, Gender

Some features like phone number or email may not be useful for prediction.

Feature selection chooses only important features such as:

Height, Weight, Age, Salary.

Types of Feature Selection

1. Filter Methods

Features are selected based on statistical measures.

Examples:

- Correlation
- Chi-square test
- Information gain

Correlation

Correlation is a statistical measure used to determine the strength and direction of the relationship between two variables. In feature selection, correlation is used to identify redundant features that

are highly related to each other. If two features have a very high correlation, they contain similar information, and one of them can be removed to reduce dimensionality. The correlation coefficient ranges from -1 to $+1$, where $+1$ indicates a strong positive relationship, -1 indicates a strong negative relationship, and 0 indicates no relationship.

Explain that it measures **relationship between features**.

Formula (optional):

$$R = \text{Cov}(X,Y) / \sigma_X \sigma_Y$$

Meaning:

- If correlation is high $\rightarrow$ features are redundant.

Chi-Square Test

The Chi-Square test is a statistical method used to measure the dependence between a feature and the class label. It compares the observed frequency of occurrences with the expected frequency if the feature and class were independent. If the difference between observed and expected values is large, the Chi-Square value becomes high, indicating that the feature is strongly related to the class and therefore useful for classification. Chi-Square is widely used in feature selection, especially for categorical data such as text classification.

Used to measure **dependence between feature and class**.

Idea:

- If feature and class are independent $\rightarrow$ feature is not useful.
- If dependence is strong $\rightarrow$ feature is important.

Formula (optional):

$$\chi^2 = \sum (\text{Observed} - \text{Expected})^2 / \text{Expected}$$

Information Gain

Information Gain is a feature selection method used to measure how much useful information a feature provides about the class label. It is based on the concept of entropy from information theory. Information gain calculates the reduction in uncertainty about the class when a particular feature is used. A feature with higher information gain is considered more important because it

helps better distinguish between classes. This method is commonly used in decision tree algorithms and text classification to identify the most informative features.

Measures **how much information a feature gives about the class**.

Based on **entropy**.

$$IG = H(\text{Class}) - H(\text{Class} \mid \text{Feature})$$

Higher information gain $\rightarrow$ better feature.

3. Wrapper Methods

Wrapper methods are a type of feature selection technique used in dimensionality reduction where a learning algorithm is used to evaluate the usefulness of different subsets of features. Instead of selecting features based only on statistical measures, wrapper methods train and test a machine learning model using various combinations of features and choose the subset that gives the best performance. These methods search through possible feature subsets using techniques such as forward selection, backward elimination, or recursive feature elimination. Although wrapper methods often provide more accurate feature selection, they are computationally expensive because the model must be trained multiple times for different feature combinations.

Examples:

- Forward selection
- Backward elimination
- Recursive feature elimination

Forward Selection

Forward selection is a feature selection technique used in wrapper methods where the process starts with no features selected. In this method, features are added one by one based on their contribution to improving the model performance. At each step, the feature that gives the best improvement in classification accuracy is added to the model. This process continues until adding more features does not significantly improve the model performance. Forward selection is useful when the number of features is large and we want to find the most important subset of features gradually.

Backward Elimination

Backward elimination is another wrapper-based feature selection method that starts with all available features in the model. In this technique, features are removed one at a time based on their importance. At each step, the feature that contributes the least to the model performance is eliminated. The model is retrained after removing each feature, and the process continues until removing additional features causes a decrease in model performance. Backward elimination is useful when we want to identify and remove irrelevant or redundant features from the dataset.

Recursive Feature Elimination (RFE)

Recursive Feature Elimination is a feature selection technique that repeatedly builds a model and removes the least important features based on the model's performance. Initially, the model is trained using all features, and the importance of each feature is evaluated. The least important feature is then removed, and the model is trained again using the remaining features. This process continues recursively until the desired number of features remains. RFE helps in selecting the most relevant features and is commonly used with machine learning algorithms such as support vector machines and regression models.

4. Embedded Methods

Embedded methods are feature selection techniques in which the process of selecting important features occurs automatically during the training of a machine learning model. Unlike filter methods that select features before training and wrapper methods that evaluate different feature subsets separately, embedded methods integrate feature selection directly into the learning algorithm. These methods identify and retain only the most relevant features while the model is being built, which improves efficiency and reduces computational cost. Examples of embedded methods include decision tree algorithms and regularization techniques such as LASSO regression, where less important features receive lower weights or are eliminated during model training. Embedded methods are effective because they combine the advantages of both filter and wrapper approaches while maintaining good predictive performance.

Feature selection occurs during model training.

Examples:

- LASSO regression
 - Decision trees
-

2. Feature Extraction

Definition

Feature extraction transforms the original features into a new set of features while reducing dimensionality.

The new features are combinations of the original ones.

Example

If original features are:

x_1, x_2, x_3

Feature extraction may create new features like:

$z_1 = a_1x_1 + a_2x_2 + a_3x_3$

$z_2 = b_1x_1 + b_2x_2 + b_3x_3$

Common Feature Extraction Methods

1. Principal Component Analysis (PCA)

- Most widely used technique.
 - Converts correlated variables into uncorrelated components.
 - Preserves maximum variance.
-

2. Linear Discriminant Analysis (LDA)

- Used for classification.
 - Maximizes separation between classes.
-

3. Singular Value Decomposition (SVD)

- Matrix factorization technique.

- Used in image processing and recommendation systems.
-

4. t-SNE

- Used for visualization of high dimensional data.
-

5. Autoencoders

- Neural network based dimensionality reduction method.
-

Difference Between Feature Selection and Feature Extraction

Feature Selection	Feature Extraction
Selects existing features	Creates new features
Original meaning preserved	Original meaning may change
Simple method	More complex method
Example: Correlation filtering	Example: PCA

Applications of Dimensionality Reduction

Dimensionality reduction is used in:

- Image recognition
 - Speech recognition
 - Text classification
 - Bioinformatics
 - Data visualization
-

Principal Component Analysis (PCA)

Principal Component Analysis (PCA) is a widely used dimensionality reduction technique in pattern recognition and machine learning. It is a feature extraction method that transforms a large set of correlated variables into a smaller set of new variables called principal components. These principal components are linear combinations of the original features and are arranged in such a way that the first principal component captures the maximum possible variance in the data, the second principal component captures the next highest variance, and so on. PCA helps reduce the dimensionality of data while preserving the most important information.

Need for PCA

In many real-world applications, datasets contain a large number of features. High-dimensional data can increase computational cost and make analysis difficult. PCA helps in reducing the number of dimensions by removing redundant or less important information while keeping the main patterns in the data. It also helps in improving visualization and reducing noise in the dataset.

Basic Idea of PCA

The basic idea of PCA is to find new axes or directions in the data where the variance is maximum. These directions are called principal components. The data is projected onto these new axes, resulting in a lower-dimensional representation that still retains most of the original information.

Steps of Principal Component Analysis

The main steps involved in PCA are as follows:

1. **Standardize the data** – Subtract the mean from each feature so that the data is centered around zero.
 2. **Compute the covariance matrix** – This matrix shows how the variables in the dataset vary with respect to each other.
 3. **Find eigenvalues and eigenvectors** – Eigenvectors determine the direction of the principal components, while eigenvalues indicate the amount of variance captured by each component.
 4. **Select principal components** – Choose the eigenvectors with the largest eigenvalues.
 5. **Project the data** – Transform the original data onto the selected principal components to obtain reduced dimensions.
-

Properties of PCA

- Principal components are orthogonal (independent) to each other.
 - The first principal component captures the maximum variance in the dataset.
 - Each subsequent component captures the remaining variance.
 - PCA reduces dimensionality without losing significant information.
-

Advantages of PCA

- Reduces computational complexity.
 - Removes redundancy in data.
 - Helps visualize high-dimensional data in 2D or 3D.
 - Improves performance of machine learning algorithms.
-

Limitations of PCA

- PCA does not consider class labels since it is an unsupervised method.
 - The new features (principal components) may not have a clear physical meaning.
 - It works best for linear relationships in the data.
-

Applications of PCA

Principal Component Analysis is used in many areas such as:

- Image compression
 - Face recognition
 - Speech recognition
 - Data visualization
 - Noise reduction in signals
-

Summary

Principal Component Analysis is an important dimensionality reduction technique that transforms original correlated features into a smaller set of uncorrelated principal components while retaining most of the data variance. It helps simplify complex datasets and improves efficiency in pattern recognition and machine learning applications.

Linear Discriminant Analysis (LDA)

Linear Discriminant Analysis (LDA) is a supervised dimensionality reduction technique used in pattern recognition and machine learning. It is mainly used for classification problems where the goal is to reduce the number of features while maintaining the maximum separation between different classes. LDA projects high-dimensional data onto a lower-dimensional space in such a way that the classes become more distinguishable from each other.

Basic Idea of LDA

The main idea of LDA is to find a projection direction that maximizes the separation between classes and minimizes the spread of data within each class. In other words, LDA tries to maximize the distance between class means while minimizing the variance within each class. This helps improve the accuracy of classification.

Steps of Linear Discriminant Analysis

1. Compute the mean of each class.
 2. Compute the within-class scatter matrix.
 3. Compute the between-class scatter matrix.
 4. Calculate the projection vector (w).
 5. Project the data onto the new axis to reduce dimensionality.
-

Advantages of LDA

- Improves class separability.
 - Reduces dimensionality of the dataset.
 - Works well for classification problems.
 - Computationally efficient.
-

Limitations of LDA

- Assumes linear boundaries between classes.
 - Works best when classes are normally distributed.
 - Performance decreases when classes overlap significantly.
-

Applications of LDA

Linear Discriminant Analysis is used in:

- Face recognition
 - Image classification
 - Speech recognition
 - Medical diagnosis
 - Pattern recognition tasks.
-

Summary

Linear Discriminant Analysis is a supervised dimensionality reduction method that finds an optimal projection direction to maximize class separability while minimizing the variance within each class. It is widely used in classification problems to improve model performance and simplify high-dimensional datasets.

Parametric Method for Density Estimation

A parametric method is a statistical technique used to estimate the probability distribution of data by assuming that the data follows a specific mathematical form or model. In this method, the distribution is described using a fixed number of parameters, and these parameters are estimated from the training data. Once the parameters are known, the probability density function can be used to classify or analyze new data points. A common example of a parametric method is the Gaussian (Normal) distribution, where the distribution is defined by two parameters: the mean (μ) and the variance (σ^2). Parametric methods are generally simple and computationally efficient, but their performance depends on how well the assumed distribution matches the actual data.

Non-Parametric Techniques for Density Estimation

Introduction

Density estimation is the process of estimating the probability distribution of a dataset. In pattern recognition, density estimation helps in understanding how data points are distributed in the feature space. Non-parametric techniques estimate the probability density directly from the data without assuming any specific mathematical form for the distribution. Unlike parametric methods, which assume a known distribution such as Gaussian, non-parametric methods are more flexible and can adapt to different shapes of data distributions.

Characteristics of Non-Parametric Methods

- No assumption about the underlying probability distribution.
 - The shape of the distribution is determined directly from the data.
 - More flexible than parametric methods.
 - Usually require a larger amount of data.
-

Types of Non-Parametric Density Estimation Methods

The main non-parametric density estimation techniques used in pattern recognition are:

1. Histogram Method
 2. Parzen Window Method
 3. k-Nearest Neighbor Density Estimation
-

1. Histogram Method

The histogram method is one of the simplest non-parametric density estimation techniques. In this method, the range of data values is divided into several equal intervals called bins. The number of data points falling into each bin is counted, and the density is estimated based on these counts. The histogram provides a graphical representation of the distribution of the data.

Steps

1. Divide the data range into equal intervals (bins).
2. Count the number of observations in each bin.
3. Estimate the density by dividing the count by the total number of samples and bin width.
4. The data range is divided into **equal intervals called bins**.

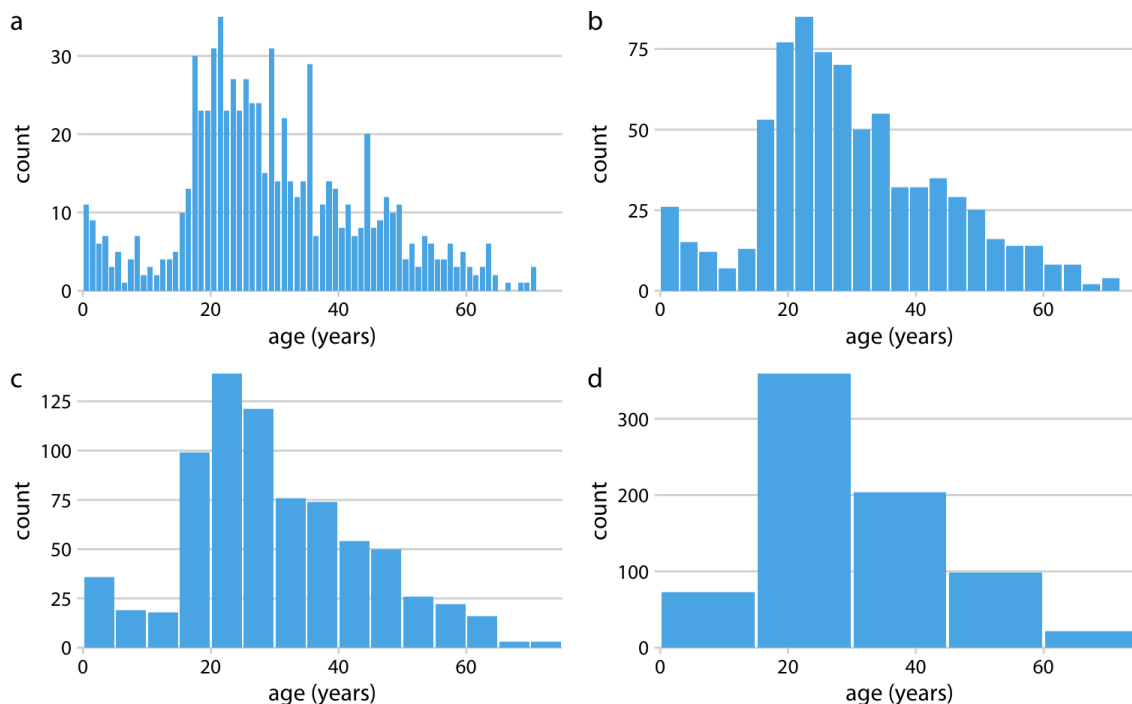
5. The number of data points in each bin is counted.
6. Density is estimated based on how many points fall in each bin.

Advantages

- Simple to understand and implement.
- Provides visual representation of data distribution.

Limitations

- The result depends heavily on bin width.
- May produce discontinuous density estimates.



2. Parzen Window Method

The Parzen Window method is a kernel-based density estimation technique. Instead of dividing the data into fixed bins like a histogram, this method places a small window or kernel function around each data point and estimates the density by averaging contributions from all points. The size of the window determines the smoothness of the density estimate.

Formula

$$p(x) = \frac{1}{nV} \sum K \left(\frac{x - x_i}{h} \right)$$

where

(n) = number of samples

(V) = volume of the window

(h) = window width

(K) = kernel function

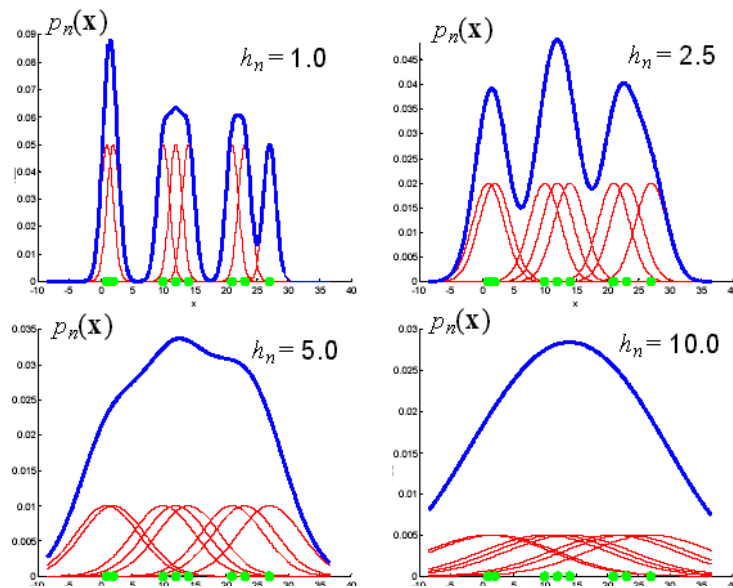
- Instead of bins, a **window (kernel)** is placed around each data point.
- Each point contributes a small smooth curve.
- All curves are added together to produce a **smooth probability density function**.

Advantages

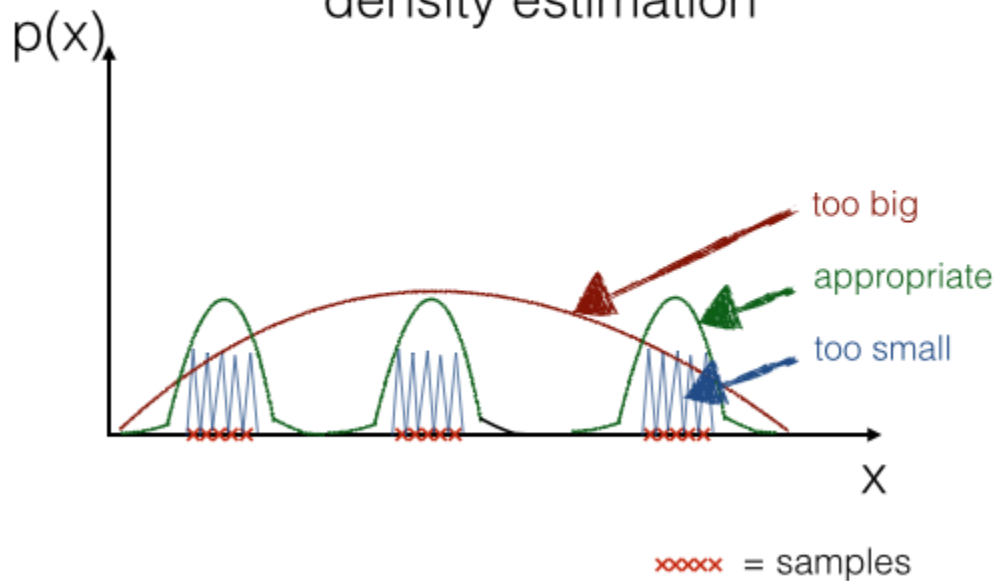
- Produces smoother density estimates than histograms.
- Flexible and adaptable to different data shapes.

Limitations

- Choosing the window size is difficult.
- Computationally expensive for large datasets.



very simplified illustration of how
the window width affects the
density estimation



Main Difference

Histogram	Parzen Window
Uses bins	Uses kernel/window
Produces step-like graph	Produces smooth curve

Histogram	Parzen Window
Sensitive to bin width	Sensitive to window size
Simple method	More accurate density estimate

3. k-Nearest Neighbor Density Estimation

In the k-Nearest Neighbor (k-NN) method, the density at a point is estimated based on the distance to its k nearest neighbors. Instead of fixing the window size like in Parzen windows, this method fixes the number of neighbors and adjusts the volume of the region accordingly.

Density Formula

$$p(x) = \frac{k}{nV}$$

where

(k) = number of nearest neighbors

(n) = total number of samples

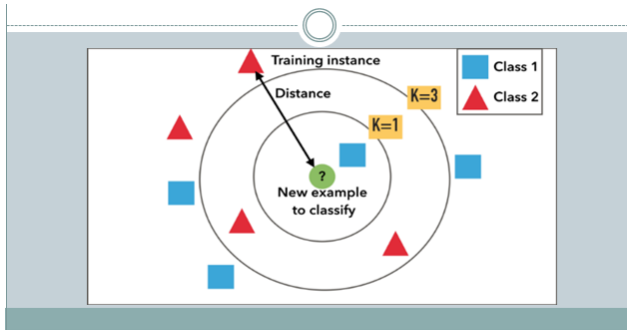
(V) = volume containing the k nearest neighbors

Advantages

- Automatically adapts to data density.
- Works well with uneven data distributions.

Limitations

- Computationally expensive for large datasets.
- Choosing the value of (k) is important.



Difference Between Parametric and Non-Parametric Methods

Parametric Methods	Non-Parametric Methods
Assume known distribution	No assumption about distribution
Few parameters to estimate	Parameters grow with data
Example: Gaussian model	Histogram, Parzen window, k-NN

Applications

Non-parametric density estimation is used in:

- Pattern recognition
 - Image processing
 - Speech recognition
 - Data analysis
 - Machine learning algorithms
-

Conclusion

Non-parametric techniques for density estimation estimate the probability distribution directly from data without assuming a specific model. Methods such as histogram, Parzen window, and k-nearest neighbor provide flexible ways to model complex data distributions in pattern recognition tasks.

Non-Metric Methods for Pattern Classification

1. Introduction

Non-metric methods for pattern classification are techniques that do not depend on distance measures (like Euclidean or Manhattan distance) or predefined mathematical metrics. Instead, these methods classify data using logical rules, decision boundaries, structural relationships, or learned models. They are useful when similarity between patterns cannot be easily measured using distance.

2. Key Idea

In non-metric methods:

- Classification is based on **rules, models, or structures**
 - No need to compute distance between data points
 - Focus is on **decision-making logic** rather than similarity
-

3. Types of Non-Metric Methods

1. Decision Tree Classification

A decision tree classifies data by applying a sequence of logical conditions.

- Each node represents a condition
- Each branch represents an outcome
- Leaf nodes represent class labels

Example:

IF marks > 50 → Pass
ELSE → Fail

2. Rule-Based Classification

In this method, classification is done using **IF–THEN rules**.

Example:

IF temperature > 35 AND humidity > 60
THEN Weather = Hot

These rules are often derived from data or expert knowledge.

3. Structural Pattern Recognition

Patterns are represented using structures like:

- Graphs
- Trees
- Strings

Instead of numerical values, relationships between components are used.

Example:

- Handwriting recognition
 - Shape recognition
-

4. Neural Network Methods

Neural networks learn classification directly from data.

- No explicit distance calculation
- Learn complex decision boundaries
- Based on training process

Example:

- Image classification
 - Speech recognition
-

5. Fuzzy Logic-Based Classification

In fuzzy methods, data can belong to multiple classes with different degrees.

Example:

A temperature can be:

- 0.7 Hot
- 0.3 Warm

Used in:

- Fuzzy C-Means clustering
 - Soft computing
-

4. Characteristics

- Do not require distance measures
 - Flexible and adaptable
 - Suitable for complex data
 - Can handle uncertainty (especially fuzzy methods)
-

5. Difference from Metric Methods

Metric Methods	Non-Metric Methods
Use distance (Euclidean, etc.)	No distance used
Example: k-NN	Example: Decision Tree
Based on similarity	Based on rules/models

6. Applications

Non-metric classification methods are used in:

- Image recognition
 - Speech processing
 - Medical diagnosis
 - Expert systems
 - Natural language processing
-

7. Conclusion

Non-metric methods classify patterns without relying on numerical distance measures. Instead, they use logical rules, structures, or learning models to make decisions. These methods are powerful for handling complex and real-world classification problems.

Unsupervised Learning

1. Introduction

Unsupervised learning is a type of machine learning in which the model is trained using data that does not have labeled outputs. In this approach, the system tries to learn patterns, structures, or relationships from the input data on its own without any prior knowledge of correct answers. It is widely used in pattern recognition to discover hidden patterns in data.

2. Key Idea

- No labeled data is provided
 - Model finds patterns automatically
 - Data is grouped based on similarity
-

3. Types of Unsupervised Learning

1. Clustering

Clustering is the process of grouping similar data points together into clusters.

Examples:

- k-Means clustering
- Hierarchical clustering
- Fuzzy C-Means

Idea:

Similar data → same group

4. Characteristics

- No supervision (no labels)
 - Finds hidden patterns
 - Works on unstructured data
 - Useful for exploratory analysis
-

5. Advantages

- No need for labeled data
 - Can discover unknown patterns
 - Useful for large datasets
 - Helps in data understanding
-

6. Limitations

- Results may not be accurate always
 - Hard to evaluate performance
 - May find meaningless patterns
 - Interpretation can be difficult
-

7. Applications

Unsupervised learning is used in:

- Customer segmentation
 - Image segmentation
 - Anomaly detection
 - Recommendation systems
 - Data compression
-

8. Difference Between Supervised and Unsupervised Learning

Supervised Learning	Unsupervised Learning
Uses labeled data	Uses unlabeled data
Output is known	Output is unknown
Example: Classification	Example: Clustering

9. Example

Suppose we have customer data without labels.
Unsupervised learning can group customers into:

- High spenders
 - Medium spenders
 - Low spenders
-

10. Conclusion

Unsupervised learning is an important approach in pattern recognition that helps in discovering hidden structures and patterns in data without using labeled outputs. It is widely used in clustering, dimensionality reduction, and data analysis tasks.

K-Means Clustering

1. Introduction

K-Means clustering is an unsupervised learning algorithm used to group similar data points into a predefined number of clusters. It partitions the dataset into **K clusters**, where each data point belongs to the cluster with the nearest mean (centroid). It is one of the most widely used clustering techniques in pattern recognition.

2. Key Idea

- Divide data into **K groups (clusters)**
- Each cluster has a **centroid (mean point)**
- Data points are assigned to the **nearest centroid**

3. Steps of K-Means Algorithm

1. **Choose number of clusters (K)**
 2. **Initialize centroids** randomly
 3. **Assign data points** to nearest centroid (using distance measure like Euclidean distance)
 4. **Update centroids** by computing mean of assigned points
 5. Repeat steps 3 and 4 until centroids do not change
-

4. Distance Formula (Euclidean Distance)

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

Used to find nearest centroid.

5. Objective Function

K-Means minimizes the **sum of squared distances** between data points and their cluster centers:

$$J = \sum ||x_i - \mu_j||^2$$

6. Example (Simple)

Data points:

(2,3), (3,4), (8,9), (9,10)

Let (K = 2)

Clusters formed:

Cluster 1 $\rightarrow$ (2,3), (3,4)

Cluster 2 $\rightarrow$ (8,9), (9,10)

7. Advantages

- Simple and easy to implement
 - Works fast for large datasets
 - Easy to understand
-

8. Limitations

- Value of K must be chosen in advance
 - Sensitive to initial centroids
 - Works best for spherical clusters
 - Affected by outliers
-

9. Applications

- Customer segmentation
- Image segmentation
- Pattern recognition
- Data compression
- Market analysis

Hierarchical Clustering

1. Introduction

Hierarchical clustering is an **unsupervised learning technique** used to group similar data points into clusters based on their similarity.

- It builds a **tree-like structure** called a **dendrogram**

- No need to specify number of clusters initially
 - Used in data mining, bioinformatics, image segmentation, etc.
-

2. Types of Hierarchical Clustering

2.1 Agglomerative (Bottom-Up Approach)

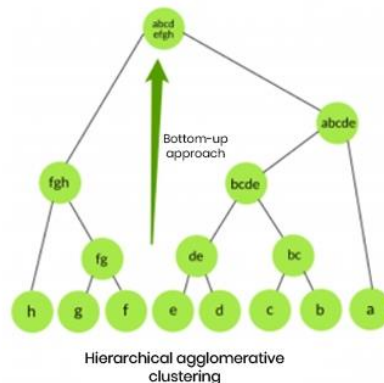
- Each data point starts as an individual cluster
- Closest clusters are merged step by step
- Continues until only one cluster remains

2.2 Divisive (Top-Down Approach)

- All data points start in one cluster
 - Recursively split into smaller clusters
 - Continues until each point becomes a separate cluster
-

3. Diagram: Agglomerative Clustering Process

Agglomerative Clustering

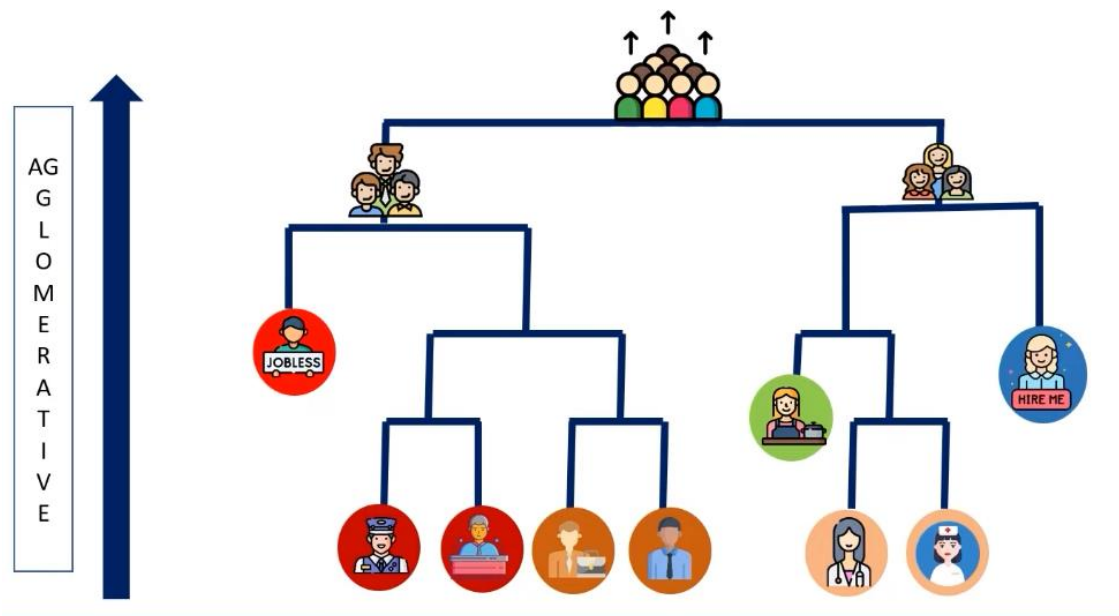


4. Dendrogram (Tree Representation)

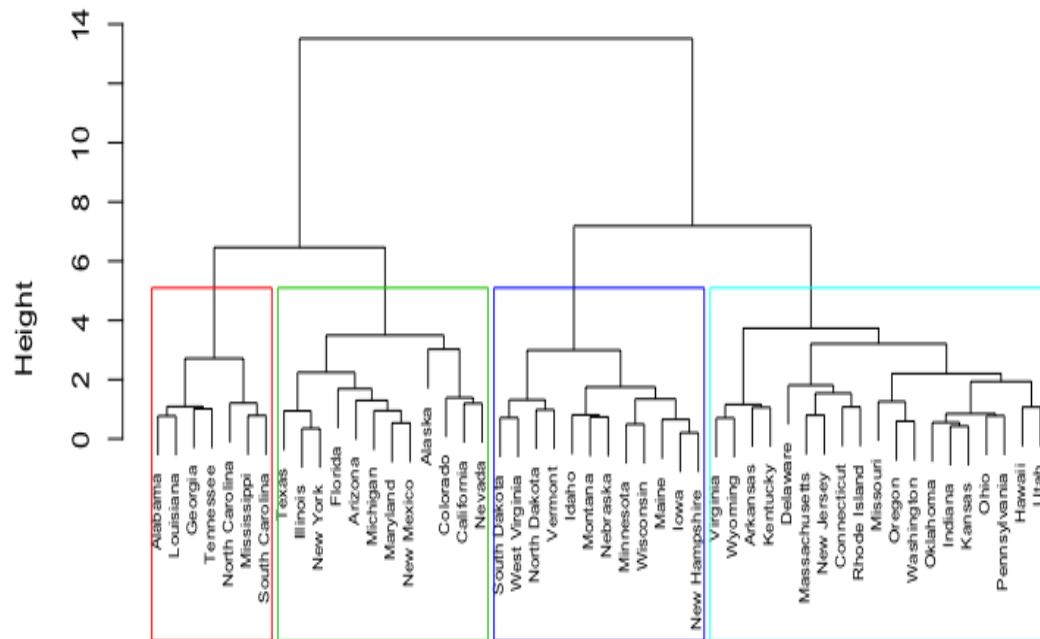
A dendrogram is a **tree diagram** used to visualize clustering.

- X-axis → Data points
- Y-axis → Distance (similarity level)
- Vertical lines show cluster merges

5. Diagram: Dendrogram



Cluster Dendrogram

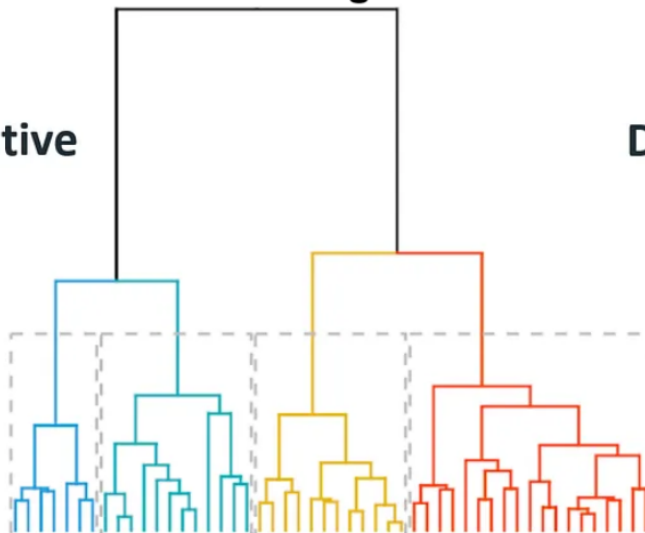


d
hclust(*, "ward.D2")

Hierarchical Clustering

Agglomerative

Divisive



6. Linkage Methods (Distance Between Clusters)

Different methods define how distance between clusters is calculated:

6.1 Single Linkage

- Distance between **closest points**
- Can form long chains (chaining effect)

6.2 Complete Linkage

- Distance between **farthest points**
- Produces compact clusters

6.3 Average Linkage

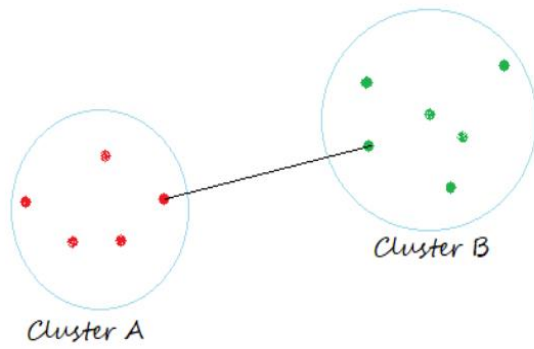
- Average distance between all points in clusters

6.4 Centroid Linkage

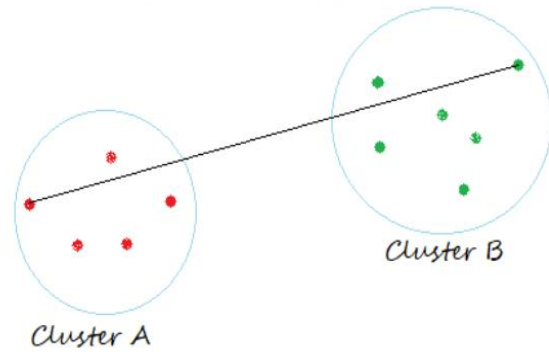
- Distance between cluster centroids

7. Diagram: Linkage Methods

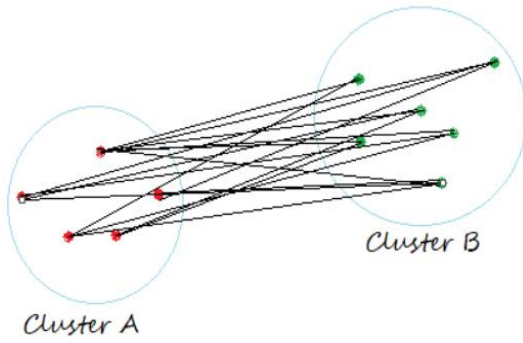
Single Linkage



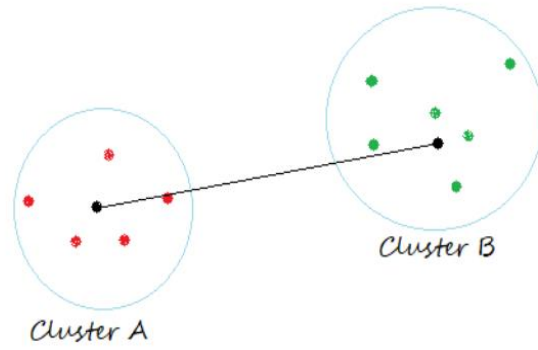
Complete Linkage

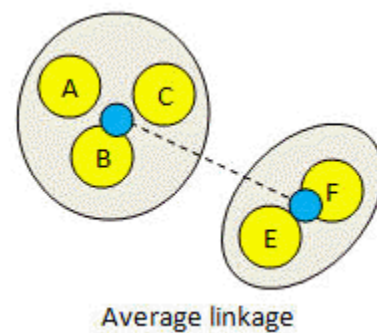
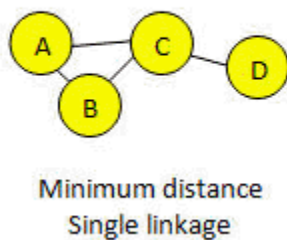
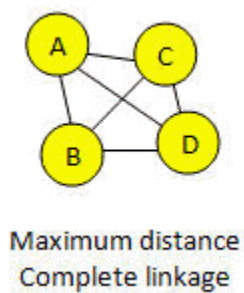
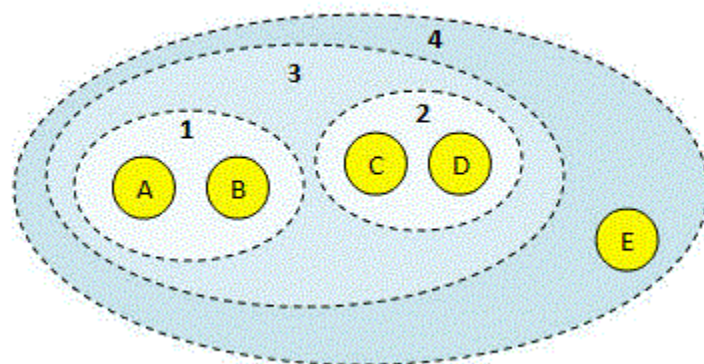
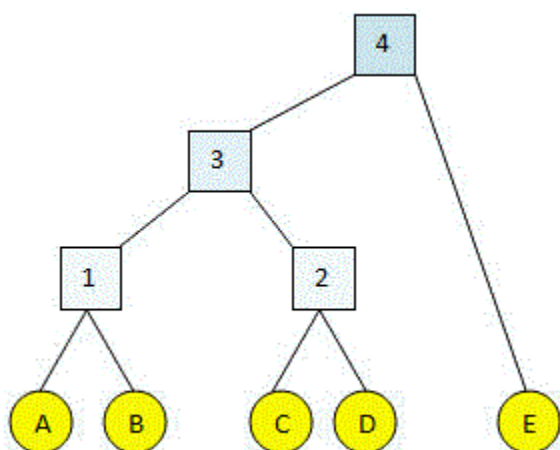
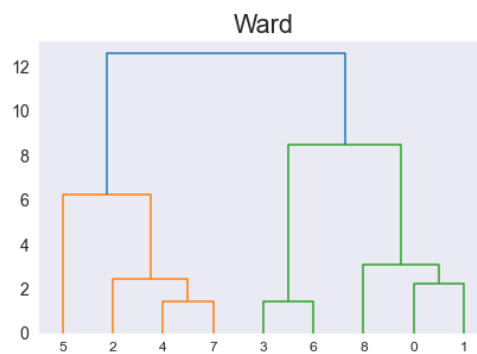
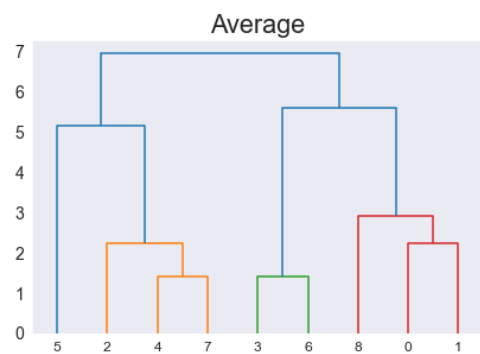
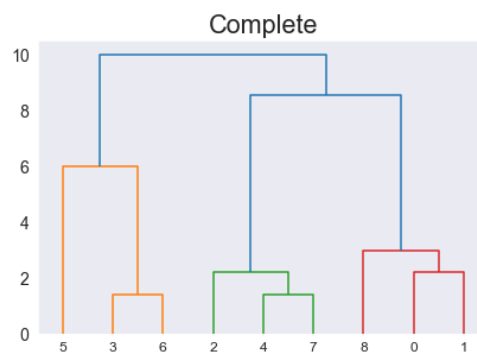
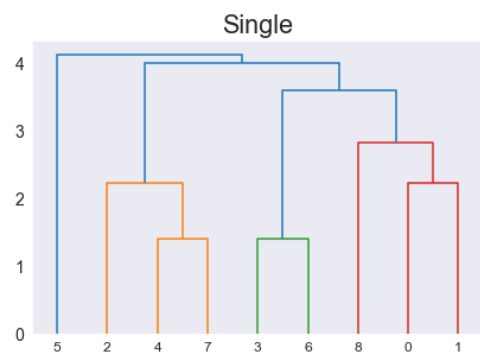


Average Linkage



Centroid Linkage





8. Algorithm (Agglomerative)

Step 1: Start with each data point as a separate cluster

Step 2: Compute distance matrix

Step 3: Merge the closest clusters

Step 4: Update distance matrix

Step 5: Repeat until one cluster remains

9. Advantages

- No need to predefine number of clusters
 - Easy to understand and interpret
 - Dendrogram provides clear visualization
-

10. Disadvantages

- Computationally expensive ($O(n^2)$ or more)
 - Not suitable for very large datasets
 - Sensitive to noise and outliers
-

11. Applications

- Gene sequence analysis
- Document clustering
- Image segmentation
- Customer segmentation